Aeronautics and Space Technology Club

IIT (ISM) Dhanbad

Signal Processing Ladder: A Hands-On Problem Series from Sensors to Machine Learning

Problem Statement 23 · Part E: Machine Learning for Signals Level 4

Track: 1D CNNs, Spectrogram CNNs, Augmentation, Learning Curves, Matched-Filter Benchmark, Calibration

Prerequisites: Problems 17, 19, 21 · **Estimated effort:** 16–22 hours

Deep Learning for Signals: Learned Filters, Honest Benchmarks

A convolutional layer is a bank of FIR filters whose taps are learned. Train a 1D CNN on raw waveforms and a 2D CNN on spectrograms to classify glitch types in noise, inspect what the first layer learned, measure how the network degrades with SNR and training-set size, and, crucially, compare it with the optimal matched-filter detector to learn where deep learning can and cannot win.

General Instructions

- **Thorough Review:** Read the whole problem statement, including the theory, before writing any code. Do the short analytic derivations *by hand first*; the code is there to confirm and extend your understanding, not to replace it.
- **Pacing:** This is a self-paced problem. The split below is a guide for the effort estimate in the header.

Stage	Phase	What you should have at the end of it
1	Think (∼20%)	Theory understood; every analytic result derived; signals, units and sampling rates written down.
2	Build (∼30%)	Core code working end to end on the baseline data and validated against the analytic results.
3	Experiment (∼30%)	Full sweeps, stress tests and the real or synthetic datasets processed; first plots and numbers.
4	Refine (∼20%)	Weaknesses fixed, honest analysis and memo written, repository folder clean.

- **Compute and Tools:** Everything is designed to run on a laptop CPU in Python 3.10+ (NumPy, SciPy, Matplotlib, and where stated pandas, Astropy, scikit-learn, PyWavelets and PyTorch). No GPU and no hardware are required; where a problem offers an optional hardware experiment it is marked as a bonus. A full run should never need more than a few minutes; if yours does, vectorise before you scale up.
- **Data:** Each problem lists (i) a *synthetic generator* from `sp_common.synth` (offline, seeded, with known ground truth) and (ii) *real datasets* to download yourself. Develop and validate on the synthetic data first, then repeat the key result on real data. Never commit large raw data files to the repository.
- **Submission Format:** Push your work to `solutions/23_deep_learning_for_signals/` in the repository: (i) clean, commented, reproducible Jupyter notebook(s) with fixed random seeds, (ii)

a short written report (`report.pdf` or `report.md`) containing the requested analysis sections, and (iii) all figures in a `figures/` sub-folder. The report must be understandable without running the code.

- **Originality:** All code, analysis and written explanations must be your own work. Any external library, snippet, paper or AI tool you use must be cited in the report. You should be able to explain every line you submit.
- **Honesty over Hype:** Validate against analytic results *before* trusting a number. A modest or negative result that is correctly analysed is worth more than an inflated claim. Report error bars, false-alarm rates and failure cases, not only your best run.
- **Units:** Write the units of every quantity and state in the first notebook cell whether a frequency is in Hz or rad/s and whether it is normalised to the sampling rate. Most mistakes in signal processing are factors of 2π, factors of 2 in one-sided spectra, or an off-by-one in the frequency axis.
- **Getting Stuck:** Re-read the theory section, check the self-check targets, then ask your mentor or open an issue in the repository. Describe what you expected, what you got, and what you already tried.

Background and Motivation

Hand-crafted features (problem 21) require you to know what distinguishes the classes. When that knowledge is incomplete, or the signals are rich (transient shapes, time–frequency tracks), a convolutional neural network can learn the features directly from raw waveforms or spectrograms. But a network is a flexible function approximator, not a source of information: if a signal of known form is buried in Gaussian noise, the matched filter of problem 19 is provably optimal, and a network can at best approach it. Where networks genuinely help is when the signal model is unknown, when noise is non-Gaussian, or when many classes with complicated structure must be separated, as with the glitch classification of gravitational-wave detectors.

This problem teaches the practice (training loops, validation, augmentation, regularisation, learning curves) and the sceptical habits that go with it: always run a strong classical baseline, test at SNRs beyond the training range, and ask what happens on inputs the network has never seen.

Theoretical Framework

1. Training

A classifier outputs class scores $\boldsymbol{z}$ and probabilities $p_c = \mathrm{e}^{z_c} / \sum_j \mathrm{e}^{z_j}$ (softmax). The cross-entropy loss over a batch is $\mathcal{L} = -\frac{1}{B} \sum_i \ln p_{y_i}(\boldsymbol{x}_i)$, minimised with minibatch gradient descent; Adam adapts the step per parameter. Watch training and validation loss: a growing gap signals overfitting; remedies are more data, augmentation, weight decay, dropout, smaller models and early stopping.

2. 1D Convolution and Receptive Field

A 1D convolutional layer with C_{in} input channels, C_{out} output channels and kernel length K computes $y_{c'}[n] = b_{c'} + \sum_c \sum_k w_{c',c}[k]\, x_c[n+k]$ (cross-correlation, as implemented in deep-learning libraries); it has $(C_{\mathrm{in}} K + 1)\, C_{\mathrm{out}}$ parameters. Each kernel is an FIR filter, and its frequency response is the FFT of w. The receptive field of a stack of layers with kernel K_ℓ and cumulative stride $s_{\ell-1}$ is

$$\mathrm{RF} = 1 + \sum_{\ell} (K_\ell - 1) \prod_{j<\ell} s_j, \tag{1}$$

which must cover the signal structure (a chirp lasting 0.5 s at 2 kHz needs RF $\gtrsim 1000$ samples, or global pooling over a feature that integrates it).

3. Spectrogram Networks

Computing a log-magnitude spectrogram (problem 17) converts the signal into an image on which a 2D CNN can use translation equivariance in time and frequency. Whitening before the transform matters for coloured noise.

4. Augmentation

Random circular time shifts, amplitude scaling, polarity flips, and added noise at random SNR encode invariances the network should have; augmentations that change the label (for instance shifting a chirp out of the window) must be avoided.

5. Optimality Limits

For H_0: noise only against H_1: known signal plus Gaussian noise, no detector beats the matched filter at a given false-alarm rate (Neyman–Pearson, problem 19). A network trained for this task approaches the matched-filter ROC from below as data and capacity grow.

6. Calibration and Out-of-Distribution Inputs

Softmax probabilities are often over-confident. Temperature scaling divides the logits by a scalar T fitted on a validation set. An input from a class never seen in training is still assigned to some class with high confidence unless an explicit rejection mechanism (a threshold on max probability or an energy score) is used.

Data and Simulation Specification

Glitch data	`sp_common.synth.glitch_zoo(n_per_class=100, fs=2048, duration=1.0, snr_range=(8,20))`: classes noise, chirp, blip, whistle in white noise; use $n = 1000$ per class for the main experiments, a held-out test set of 2000 generated with a different seed and an SNR sweep set at fixed SNR $= 4, 6, \ldots, 30$.
Binary detection set	Chirp versus noise only, same generator; for the matched-filter comparison use the exact chirp templates (`newtonian_chirp` with a bank over chirp mass) and `scipy.signal.correlate`.
Bearing data	`sp_common.synth.bearing_dataset` with group split (problem 21).
Framework	PyTorch (CPU is sufficient: models of fewer than 10^5 parameters train in minutes). Fix all seeds (`torch.manual_seed`) and report seed-to-seed variation over at least five seeds.
Real data (choose one)	Gravity Spy glitch images and metadata from the Zenodo record linked at `gravityspy.org` (spectrogram classification of real interferometer glitches); or CWRU bearing segments as in problem 21.

Your Task

1. **1D CNN (Stage 2).** Build a small 1D CNN (for example three blocks of Conv1d–BatchNorm–ReLU–MaxPool, then global average pooling and a linear layer). Train with Adam on the glitch data with a train/validation split, plot learning curves, report test accuracy and confusion matrix, and repeat over five seeds to give mean and standard deviation. Count parameters by

hand with the formula in Section 2 and check with `sum(p.numel())`.

2. **Spectrogram CNN.** Compute log-spectrograms (window and hop chosen with the rule of problem 17), train a 2D CNN and compare accuracy, number of parameters and training time with the raw-waveform network. Test the effect of whitening and of the spectrogram window length.
3. **Receptive field and learned filters.** Compute the receptive field of your 1D network with Eq. (1). Plot the first-layer kernels and their FFT magnitudes and describe what bands they select. Find, by shrinking the architecture, the smallest network that reaches 95% accuracy and relate its receptive field to the signal duration.
4. **Robustness and the matched-filter benchmark.** Evaluate accuracy versus SNR (test set at fixed SNR from 4 to 30) for networks trained on SNR 8–20; then train and test a binary chirp-versus-noise network and compare its ROC (at equal false-alarm rate, with Wilson intervals) with the matched-filter bank. State quantitatively how close the network gets to the optimum at SNR = 6, 8, 10 and where it is far from it.
5. **Augmentation.** Train with time shift, amplitude scaling and noise augmentation and without; quantify accuracy on SNR outside the training range and on time-shifted test samples.
6. **Data efficiency and overfitting.** Train on $10, 30, 100, 300, 1000$ samples per class; plot the learning curve for the network and for the random forest on features (problem 21). Vary dropout and weight decay and show the effect on the gap between training and validation accuracy. Find the data size at which the network overtakes the feature-based model, if it does.
7. **Explainability, calibration and out-of-distribution.** Compute occlusion or gradient saliency maps for correctly classified examples and verify the network looks where the signal is. List and diagnose misclassified examples. Compute the expected calibration error before and after temperature scaling. Feed a new signal class (for example a sine burst or a frequency-modulated pulse) and measure the confidence of the (wrong) predictions; implement a rejection threshold and evaluate it.
8. **Real data and cost.** Train on your real dataset (or on the bearing data with group splits) and compare with the classical pipeline of problem 21. Report parameters, multiply–accumulate operations per prediction (compute by hand) and CPU inference time per sample.

Self-check targets

A random guess on four balanced classes gives 25%. A `Conv1d` layer with $C_{\text{in}} = 8$, $C_{\text{out}} = 16$, $K = 9$ has $(8 \cdot 9 + 1) \cdot 16 = 1168$ parameters. For three blocks of a kernel-9 convolution followed by max-pooling (kernel 2, stride 2), Eq. (1) gives $\text{RF} = 1 + 8 + 1 + 16 + 2 + 32 + 4 = 64$ samples. On the binary detection task the matched filter must lie on or above every network ROC curve (apart from sampling error).

Aerospace Application Link

A CubeSat has a CPU of a few hundred MHz and a power budget of a watt; an FPGA payload may run an int8 network. Using your parameter and operation counts state whether the network fits (memory in kilobytes at 32-bit and at int8, operations per second at one prediction per second per channel), what accuracy you lose when the weights are quantised to 8 bits (measure it by rounding your trained weights), and what happens when the on-orbit noise is not the Gaussian noise of the training set. Argue for or against flying the network instead of the matched-filter bank.

Deliverables

D1 **1D CNN:** code, learning curves, confusion matrix, five-seed statistics and the parameter count check.

D2 **Spectrogram CNN and receptive field:** comparison table, learned-filter plots, minimal architecture.

D3 **Robustness and benchmark:** accuracy-versus-SNR plot and the network-versus-matched-filter ROC comparison.

D4 **Augmentation and data efficiency:** the ablation and the learning curves versus the feature-based model.

D5 **Explainability, calibration, out-of-distribution, real data and cost table.**

D6 **Memo (300 words):** "When should we fly a neural network instead of a matched filter?" Cite at least two external sources.

Bonus (a) Quantise the network to int8 with post-training quantisation and measure the loss in accuracy. (b) Add a residual connection and a dilated convolution and compare receptive fields. (c) Train a small transformer encoder on the spectrogram patches and compare data efficiency.

Learning Resources

Deep Learning

- Goodfellow, I., Bengio, Y., Courville, A. *Deep Learning*, Ch. 9 (convolutional networks), Ch. 7 (regularisation), Ch. 8 (optimisation).
- Kingma, D.P., Ba, J. (2015). "Adam: a method for stochastic optimization." arXiv:1412.6980.
- Guo, C., Pleiss, G., Sun, Y., Weinberger, K.Q. (2017). "On calibration of modern neural networks." Proc. ICML, arXiv:1706.04599.
- PyTorch tutorials: `https://pytorch.org/tutorials/`

Time-Series and Signal Classification

- Ismail Fawaz, H., Forestier, G., Weber, J., Idoumghar, L., Muller, P.-A. (2019). "Deep learning for time series classification: a review." Data Min. Knowl. Discov. 33, 917, arXiv:1809.04356.
- Zevin, M. et al. (2017). "Gravity Spy: integrating Advanced LIGO detector characterization, machine learning, and citizen science." Class. Quantum Grav. 34, 064003.
- George, D., Huerta, E.A. (2018). "Deep learning for real-time gravitational wave detection and parameter estimation." Phys. Lett. B 778, 64, arXiv:1711.03121.

Data and Tools

- Gravity Spy: `https://gravityspy.org`; CWRU Bearing Data Center: `https://engineering.case.edu/bearingdatacenter`

Evaluation

Score your own submission out of 100 on five equally weighted criteria, then ask your mentor to score it independently and compare.

Criterion	What to look for	Marks
Conceptual depth	Correct mathematics and physics; every result explained through a mechanism (why the spectrum looks like this), not only through code behaviour.	/20
Implementation quality	Correct, modular, commented, reproducible code; no unexplained magic numbers; analytic checks and self-check targets pass.	/20
Aerospace-application integration	The signal-processing result is carried through to a measurable consequence for a sensor, a link, a flight system or a detection decision, as the task requires.	/20
Analysis and critical thinking	Error bars, convergence, comparison to theory, honest treatment of limitations, false alarms and failure cases.	/20
Communication	Clear report, labelled plots (units, legends, captions) and a memo that a non-expert student could follow.	/20

Bonus tasks are not part of the 100 marks. A bonus attempted badly does not cost you marks, and a well-executed one is the best sign that you own the material.